SOURCE CODE

Source Code Documentation

Implementation Details • Database Architecture • Frontend Integration

Prepared for Project Review

Contents

• 1. Google Gemini AI Advisory Engine with Automatic Retry and Rule-Based Fallback

• 2. Server-Side Financial Calculation and Analytics Engine

• 3. Relational Database Models with Foreign Key Isolation

• 4. Client-Side Indian Rupee (INR) Formatter and Asynchronous API Communication

• 5. Modular Application Factory and Blueprint Registration

• 6. Report and Presentation Placement Guide

1. Google Gemini AI Advisory Engine with Automatic Retry and Rule-Based Fallback

File: services/gemini_service.py

Purpose: Sends ledger metrics to Gemini and requests structured JSON insights. If the API key is missing or calls fail, the service is intended to return a local rule-based fallback analysis.

```
import json
import logging
import os
import re
from typing import Dict, Any
logger = logging.getLogger(__name__)
DISCLAIMER_TEXT = (
"AI-generated information is for educational and planning purposes and "
"should not be considered professional financial advice."
)
class GeminiService:
"""Service wrapper for Google Gemini API integration."""
@classmethod
def get_api_key(cls) -> str:
return os.environ.get("GEMINI_API_KEY", "").strip()
@classmethod
def get_model_name(cls) -> str:
return os.environ.get("GEMINI_MODEL", "gemini-3.8-flash").strip()
@classmethod
def analyze_finances(cls, financial_data: Dict[str, Any]) -> Dict[str, Any]:
"""Send user financial metrics to Gemini and return structured insights."""
api_key = cls.get_api_key()
if not api_key or api_key == "your_gemini_api_key_here":
return cls._build_fallback_analysis(
financial_data,
reason="GEMINI_API_KEY is not configured in .env. Showing local baseline calculation.",
)
prompt = cls._build_prompt(financial_data)
# Dual-attempt call with backoff for transient 503 capacity spikes
for attempt in range(2):
try:
from google import genai
from google.genai import types
client = genai.Client(api_key=api_key)
model_name = cls.get_model_name()
response = client.models.generate_content(
model=model_name,
contents=prompt,
config=types.GenerateContentConfig(
temperature=0.2,
max_output_tokens=2048,
),
)
raw_text = response.text or ""
parsed = cls._parse_ai_response(raw_text, financial_data)
parsed["source"] = "gemini"
parsed["disclaimer"] = DISCLAIMER_TEXT
return parsed
except Exception as e:
logger.warning(f"Gemini call attempt {attempt+1} failed: {e}")
if attempt == 0:
import time
time.sleep(1.5)
else:
return cls._build_fallback_analysis(
financial_data,
reason=f"Gemini API status notice: {str(e)[:120]}. Displaying rule-based financial analysis.",
)
@classmethod
def _build_prompt(cls, data: Dict[str, Any]) -> str:
"""Construct structured prompt specifying Indian Rupees (INR, ■)."""
income = data.get("total_income", 0.0)
expenses = data.get("total_expenses", 0.0)
savings = data.get("net_savings", 0.0)
savings_pct = data.get("savings_percentage", 0.0)
categories = data.get("category_breakdown", {})
budget_comp = data.get("budget_comparison", [])
goals = data.get("savings_goals", {}).get("items", [])
cat_summary = ", ".join(
[f"{k}: ■{v.get('amount', 0):,.2f}" for k, v in categories.items()]
) or "No expenses"
budget_str = "; ".join(
[f"{b.get('category')}: Limit ■{b.get('limit', 0):,.0f}, Spent ■{b.get('spent', 0):,.0f}" for b in budget_comp]
```

```
) or "No budgets"
goals_str = "; ".join(
[f"{g.get('name')}: Saved ■{g.get('current_amount', 0):,.0f}/■{g.get('target_amount', 0):,.0f}" for g in goals]
) or "No goals"
return f"""You are a personal finance analysis assistant. Analyze the user's monthly ledger in Indian Rupees (INR, ■).
Rules:
1. Provide actionable advice grounded in practical financial management (50/30/20 rule, emergency funds).
2. Do not use fake emojis, vague AI cliches, or em dashes.
3. Output valid JSON only matching the schema below.
User Summary:
- Monthly Income: ■{income:,.2f}
- Monthly Expenses: ■{expenses:,.2f}
- Net Monthly Savings: ■{savings:,.2f} ({savings_pct}% rate)
- Categories: {cat_summary}
- Budgets: {budget_str}
- Goals: {goals_str}
Respond with ONLY valid JSON:
{{
"monthly_financial_summary": "2-3 sentences summarizing cashflow status.",
"spending_insights": ["Insight 1", "Insight 2", "Insight 3"],
"overspending_categories": [{{"category": "Name", "issue": "Explanation", "severity": "high/medium"}}],
"budget_recommendations": ["Recommendation 1", "Recommendation 2"],
"saving_suggestions": ["Action 1", "Action 2"],
"unnecessary_expense_reductions": ["Reduction 1", "Reduction 2"]
}}"""
```

2. Server-Side Financial Calculation and Analytics Engine

File: services/finance_service.py

Purpose: Calculates monthly income, expenses, net savings, savings percentage, category totals, and budget utilization on the server.

```
from datetime import date
from sqlalchemy import func, extract
from models import db, Income, Expense, Budget, SavingsGoal
class FinanceService:
"""Service layer handling financial calculations, validations, and data aggregation."""
@classmethod
def get_user_dashboard_summary(cls, user_id: int, month_str: str = None) -> dict:
"""Aggregate full financial dashboard statistics for authenticated user."""
if not month_str:
today = date.today()
year_int, month_int = today.year, today.month
month_str = f"{year_int:04d}-{month_int:02d}"
else:
year_int, month_int = map(int, month_str.split("-"))
# 1. Total Monthly Income
monthly_income_sum = (
db.session.query(func.coalesce(func.sum(Income.amount), 0.0))
.filter(
Income.user_id == user_id,
extract("year", Income.date) == year_int,
extract("month", Income.date) == month_int,
)
.scalar()
)
total_income = float(monthly_income_sum)
# 2. Total Monthly Expenses
monthly_expense_sum = (
db.session.query(func.coalesce(func.sum(Expense.amount), 0.0))
.filter(
Expense.user_id == user_id,
extract("year", Expense.date) == year_int,
extract("month", Expense.date) == month_int,
)
.scalar()
)
total_expenses = float(monthly_expense_sum)
# 3. Net Savings & Savings Percentage
net_savings = total_income - total_expenses
savings_percentage = (
round((net_savings / total_income) * 100.0, 2)
if total_income > 0 else 0.0
)
# 4. Category-Wise Expenses
category_rows = (
db.session.query(
Expense.category,
func.sum(Expense.amount).label("total"),
)
.filter(
Expense.user_id == user_id,
extract("year", Expense.date) == year_int,
extract("month", Expense.date) == month_int,
)
.group_by(Expense.category)
.all()
)
category_breakdown = {
r.category: {"amount": float(r.total)} for r in category_rows
}
# 5. Budget vs Actual Comparison with Alert Thresholds
budgets = Budget.query.filter_by(user_id=user_id, month=month_str).all()
budget_map = {b.category: float(b.limit) for b in budgets}
budget_comparison = []
for cat, limit in budget_map.items():
spent = category_breakdown.get(cat, {}).get("amount", 0.0)
pct_used = round((spent / limit) * 100.0, 2) if limit > 0 else 0.0
status = (
```

```
"exceeded" if pct_used >= 100.0
else ("warning" if pct_used >= 80.0 else "on_track")
)
budget_comparison.append({
"category": cat,
"limit": limit,
"spent": spent,
"remaining": limit - spent,
"percentage_used": pct_used,
"status": status,
})
return {
"total_income": total_income,
"total_expenses": total_expenses,
"net_savings": net_savings,
"savings_percentage": savings_percentage,
"category_breakdown": category_breakdown,
"budget_comparison": budget_comparison,
}
```

3. Relational Database Models with Foreign Key Isolation

File: models/user.py and models/savings_goal.py

Purpose: Defines user and savings-goal database models, password hashing, user-linked records, and savings-goal progress calculation.

```
from datetime import datetime, timezone
from werkzeug.security import generate_password_hash, check_password_hash
from . import db
class User(db.Model):
"""User account model with PBKDF2/SHA-256 salted password hashing."""
__tablename__ = "users"
id = db.Column(db.Integer, primary_key=True)
name = db.Column(db.String(100), nullable=False)
email = db.Column(db.String(120), unique=True, nullable=False, index=True)
password_hash = db.Column(db.String(255), nullable=False)
created_at = db.Column(
db.DateTime,
default=lambda: datetime.now(timezone.utc),
)
# Cascade delete isolates all user records
incomes = db.relationship(
"Income", backref="user", cascade="all, delete-orphan", lazy=True
)
expenses = db.relationship(
"Expense", backref="user", cascade="all, delete-orphan", lazy=True
)
budgets = db.relationship(
"Budget", backref="user", cascade="all, delete-orphan", lazy=True
)
savings_goals = db.relationship(
"SavingsGoal", backref="user", cascade="all, delete-orphan", lazy=True
)
def set_password(self, password: str):
self.password_hash = generate_password_hash(password)
def check_password(self, password: str) -> bool:
return check_password_hash(self.password_hash, password)
class SavingsGoal(db.Model):
"""Savings target milestone with progress calculation."""
__tablename__ = "savings_goals"
id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(
db.Integer,
db.ForeignKey("users.id", ondelete="CASCADE"),
nullable=False,
)
name = db.Column(db.String(100), nullable=False)
target_amount = db.Column(db.Float, nullable=False)
current_amount = db.Column(db.Float, nullable=False, default=0.0)
target_date = db.Column(db.Date, nullable=False)
@property
def progress_percentage(self) -> float:
if self.target_amount <= 0:
return 0.0
return round((self.current_amount / self.target_amount) * 100.0, 2)
```

4. Client-Side Indian Rupee (INR) Formatter and Asynchronous API Communication

File: static/js/app.js

Purpose: Formats amounts in the Indian numbering system and provides a reusable asynchronous API helper with login redirection for HTTP 401 responses.

```
/**
* Formats numeric values into the standard Indian Rupee format
* (e.g. ■1,50,000.00).
*/
function formatINR(amount) {
if (amount === undefined || amount === null || isNaN(amount)) {
return "■0.00";
}
const isNegative = amount < 0;
```

```
const absAmount = Math.abs(Number(amount));
const parts = absAmount.toFixed(2).split(".");
let intPart = parts[0];
const decPart = parts[1];
let formattedInt = "";
if (intPart.length <= 3) {
formattedInt = intPart;
} else {
const last3 = intPart.substring(intPart.length - 3);
const rest = intPart.substring(0, intPart.length - 3);
const chunks = [];
let r = rest;
while (r.length > 2) {
chunks.unshift(r.slice(-2));
r = r.slice(0, -2);
}
if (r.length > 0) chunks.unshift(r);
formattedInt = chunks.join(",") + "," + last3;
}
return (isNegative ? "-■" : "■") + formattedInt + "." + decPart;
}
/**
* Universal async API client with 401 unauthenticated redirect.
*/
async function apiFetch(url, options = {}) {
const defaultHeaders = {
"Content-Type": "application/json",
Accept: "application/json",
};
options.headers = { ...defaultHeaders, ...options.headers };
const response = await fetch(url, options);
if (response.status === 401) {
window.location.href = "/login";
return null;
}
const data = await response.json().catch(() => ({}));
if (!response.ok) {
throw new Error(data.error || `HTTP error ${response.status}`);
}
return data;
}
```

5. Modular Application Factory and Blueprint Registration

File: app.py

Purpose: Creates the Flask application, loads configuration, initializes extensions, registers route blueprints, and creates database tables.

```
import os
from flask import Flask, jsonify, render_template
from flask_cors import CORS
from config import Config
from models import db
from routes.pages import pages_bp
from routes.auth import auth_bp
from routes.dashboard import dashboard_bp
from routes.income import income_bp
from routes.expenses import expenses_bp
from routes.budgets import budgets_bp
from routes.goals import goals_bp
from routes.ai import ai_bp
def create_app(config_class=Config) -> Flask:
"""Application factory initializing extensions, blueprints, and database tables."""
app = Flask(__name__)
app.config.from_object(config_class)
CORS(app, supports_credentials=True)
db.init_app(app)
# Register modular blueprints
app.register_blueprint(pages_bp)
app.register_blueprint(auth_bp)
app.register_blueprint(dashboard_bp)
app.register_blueprint(income_bp)
app.register_blueprint(expenses_bp)
app.register_blueprint(budgets_bp)
app.register_blueprint(goals_bp)
app.register_blueprint(ai_bp)
with app.app_context():
db.create_all()
return app
app = create_app()
if __name__ == "__main__":
port = int(os.environ.get("PORT", 5000))
app.run(host="127.0.0.1", port=port, debug=True)
```

6. Report and Presentation Placement Guide

Implementation Details (Chapters 7 and 9): Use Topics 1 and 2 to explain the AI integration, retry/fallback approach, and server-side financial calculations.

Database Architecture (Chapter 10): Use Topic 3 to explain SQLAlchemy models, password hashing, foreign keys, and relationships between users and their records.

Frontend Integration (Chapter 11): Use Topic 4 to explain Indian currency formatting and asynchronous communication with backend APIs.

Presentation Slides: Focus on Topics 1 and 2, showing how the AI advisory component and financial calculation service work behind the scenes.

Important Implementation Notes

The supplied snippets refer to helper methods such as _build_fallback_analysis and _parse_ai_response that are not included in the provided excerpt. Include their implementations in the final project source if they are required by the application.

The finance-service description mentions a rolling six-month historical trend, but the supplied code excerpt does not implement that trend query. Add the relevant query before describing that feature as implemented.

For production deployment, avoid running Flask with debug=True and restrict CORS to trusted origins. Keep GEMINI_API_KEY in environment configuration and never place the secret key directly in source code.